

NTI - DIGITAL IC DESIGN USING FPGA

FINAL PROJECT: UART (TX PART)

INSTRUCTOR: ENG. MOHAMMED EL-SAYED

MADE BY: AHMED MOSLEH

Digital IC DESIGN USING FPGA FINAL
PROJECT
UART (TX-PART)

احمد محمد مصلح فهمي الجندي

الاسم:

336308

كود

التسجيل:

Introduction:

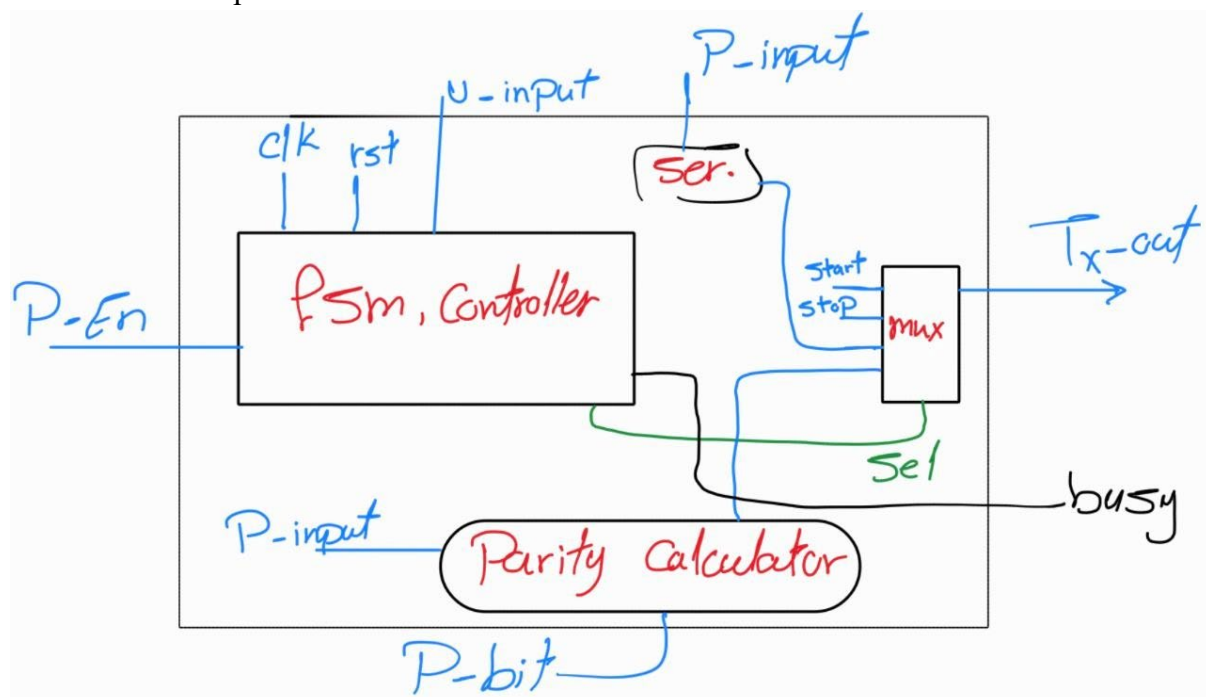
In this Project we will discuss what is a UART (specially its TX-Part), explain how it works in hardware, its implementation in **Verilog**, and its **Schematics** and **Time Analysis**.

So, what is a UART?

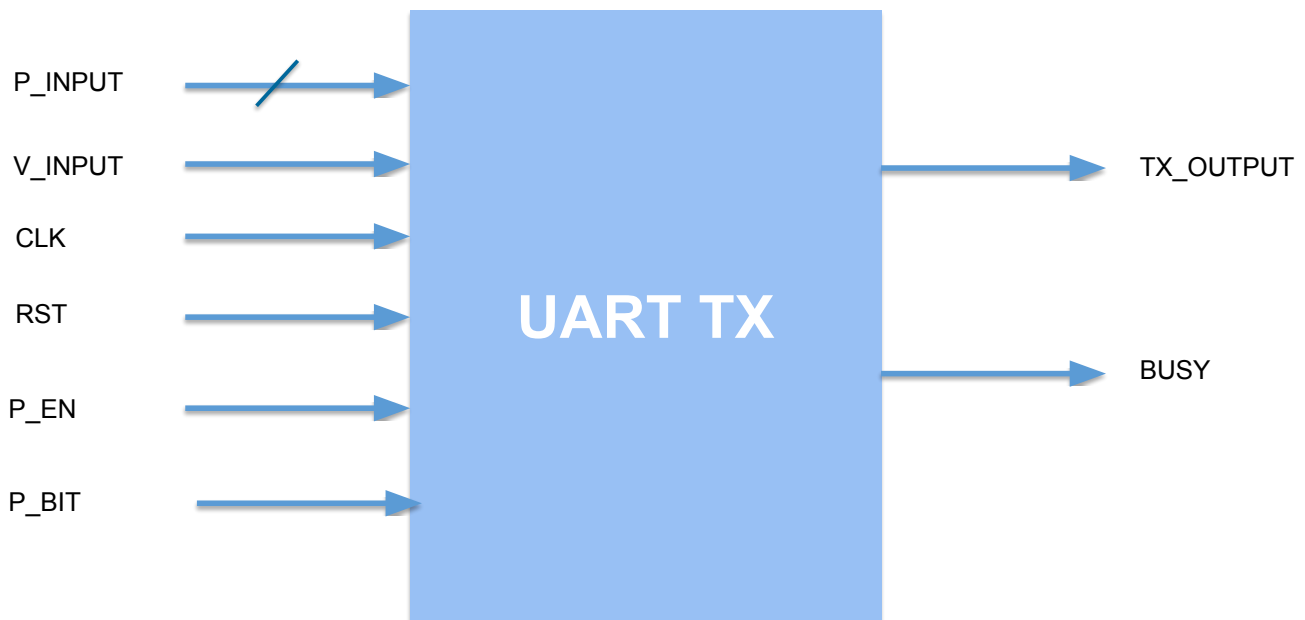
UART (Universal Asynchronous Receiver-Transmitter) is a serial communication protocol used for transmitting data between devices without a shared clock signal. It is designed to convert an 8-bit parallel input into a structured 10-bit serial frame ready for transmission from a Transmitter (TX). That's the part that we will discuss in the current time, to a Receiver (RX), will be discussed in the future In Shaa Allah.

How does the TX part work?

The general idea is transmitting a start bit, to let the receiver know that we are sending data, while passing the 8-bit parallel data into a serializer to turn it into serial data, and checking the polarity of the data (if required) to send a polarity bit and ending the transmission by sending a stop bit. All this process is organized by a Finite State Machine and filtered through a Mux for the output.



TX Frame



The ports of the TX consist of:

Input:

P_INPUT → This is the port that will have the 8-bit parallel data that will go to the Serializer.

V_INPUT → Tells us that the input is **valid** and ready for transmission.

CLK → System clock.

RST → Asynchronous active-low reset signal to clear the registers.

P_EN → Parity Enable Control (1: Enable, 0: Disable).

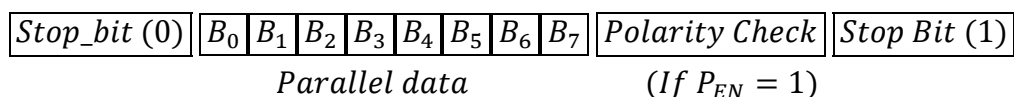
P_BIT → Parity Bit Selection (0: Even Parity, 1: Odd Parity).

Output:

TX_OUTPUT → Serial Data Output (Also contains Start bit, Polarity Check and Stop bit).

BUSY → Status signal that goes high (1) during transmission and low (0) otherwise.

The output data should be like this:



To make the TX we need Four Modules which are:

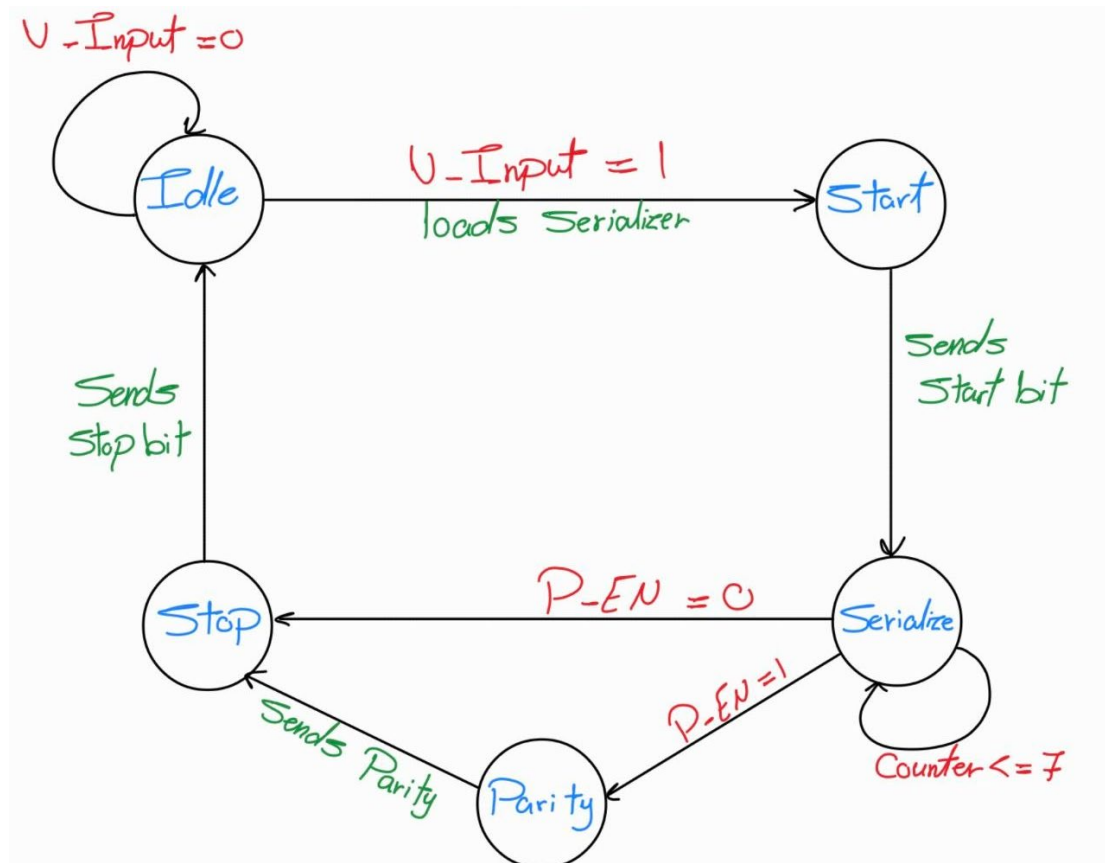
1. Finite State Machine Controller (FSM Controller)
2. Parallel to Series Shift Register (Serializer)
3. Polarity Bit Calculator
4. Mux

Let's talk about each module in detail.

1. Finite State Machine Controller (FSM Controller):

Also called the brain of the Top Module (TX), which is a Five-State FSM that is responsible for every step in the Top Module (TX), and sending signals to the other module telling them when to start, and it sends both the start bit and the stop bit in their right place, and the sets the **BUSY** high. This ensures that the circuit follows a chronological order for each bit sent.

FSM design:



1. **Idle State** → The System waits for V_INPUT and transmission lines stay high (1). From Idle to Start → FSM sends a signal to the serializer to start the loading process.
2. **Start State** → Transmits the Start bit (0) in the TX_OUPUT and the FSM tells the serializer to start serializing the data.
3. **Serialize State** → The Serializer serializes the data and sends it to the TX_OUTPUT one by one.
After this state the next state depends on P_EN
$$\text{Next state} \begin{cases} \text{Parity State,} & \text{if } P_EN = 1 \\ \text{Stop State,} & \text{if } P_EN = 0 \end{cases}$$
4. **Parity State** → The FSM sends a signal to the Parity Bit Calculator to check the parity of the data then it sends it in the TX_OUTPUT.
5. **Stop State** → This is the final state where everything is sent to the TX_OUTPUT, and the last thing is to send the Stop bit (1) through the TX_OUTPUT.

The Verilog Code:

```
module FSM_Controller (
input clk, rst, P_EN, V_INPUT,
output reg Start_Bit, Serial_start, Serial_load, Stop_Bit, busy, Enable,
output reg [1:0] sel
);

reg [3:0] counter, current, next;

localparam S0 = 3'b000,
           S1 = 3'b001,
           S2 = 3'b010,
           S3 = 3'b011,
           S4 = 3'b100;

always @(posedge clk or negedge rst)
begin
    if (!rst)
        begin
            current <= S0;
            counter <= 4'd0;
        end

    else
        begin
            current <= next;

            if (current == S2)
                begin
                    counter <= counter + 4'd1;
                end

            else
                begin
                    counter <= 4'd0;
                end
        end
    end
end

// start(0) b0 b1 b2 b3 b4 b5 b6 b7 stop(1)

always @(*)
begin
    next = current;
    Serial_load = 0;
    Serial_start = 0;
    Start_Bit = 0;
    busy = 0;
    Enable = 0;
    Stop_Bit = 1;
    sel = 2'b11;

    case (current)
        S0 : begin
            if (V_INPUT)
                begin
                    next = S1;
                    Serial_load = 1;
                    busy = 1;
                end
            else
                begin

```

```

        sel = 2'b11;
        next = S0;
        Serial_load = 0;
        busy = 0;
    end
end

S1 : begin
    sel = 2'b00;
    next = S2;
    Serial_start = 1;
    busy = 1;
end

S2 : begin
    sel = 2'b01;
    if (counter >= 4'd7)
        begin
            if (P_EN)
                begin
                    next = S3;
                    Enable = 1;
                end
            else
                begin
                    next = S4;
                end
            end
        end
    else
        begin
            next = S2;
        end
    end
    busy = 1;
end

S3 : begin
    sel = 2'b10;
    Enable = 1;
    busy = 1;
    next = S4;
end

S4 : begin
    sel = 2'b11;
    next = S0;
end

default : begin
    next = S0;
end

endcase
end

endmodule

```

Test Bench Code (Verilog):

```
`timescale 1ns/1ps
module FSM_Controller_TB;

reg clk, rst, P_EN, V_INPUT;
wire Start_Bit, Serial_start, Serial_load, Stop_Bit, busy, Enable;
wire [1:0] sel;

FSM_Controller dut ( .clk(clk), .rst(rst), .P_EN(P_EN), .V_INPUT(V_INPUT), .Start_Bit(Start_Bit),
.Serial_start(Serial_start), .Serial_load(Serial_load),
                    .Stop_Bit(Stop_Bit), .busy(busy), .Enable(Enable), .sel(sel)
);

integer i;

always #10 clk = ~clk;

initial
begin
    $dumpfile ("FSM_Controller_TB.dmp");
    $dumpvars (0, FSM_Controller_TB);

    clk = 0;
    rst = 1;
    P_EN = 0;
    V_INPUT = 0;

    #20;
    rst = 0;
    #20;
    rst = 1;
    #20;

    for (i = 0 ; i < 4 ; i = i + 1)
    begin

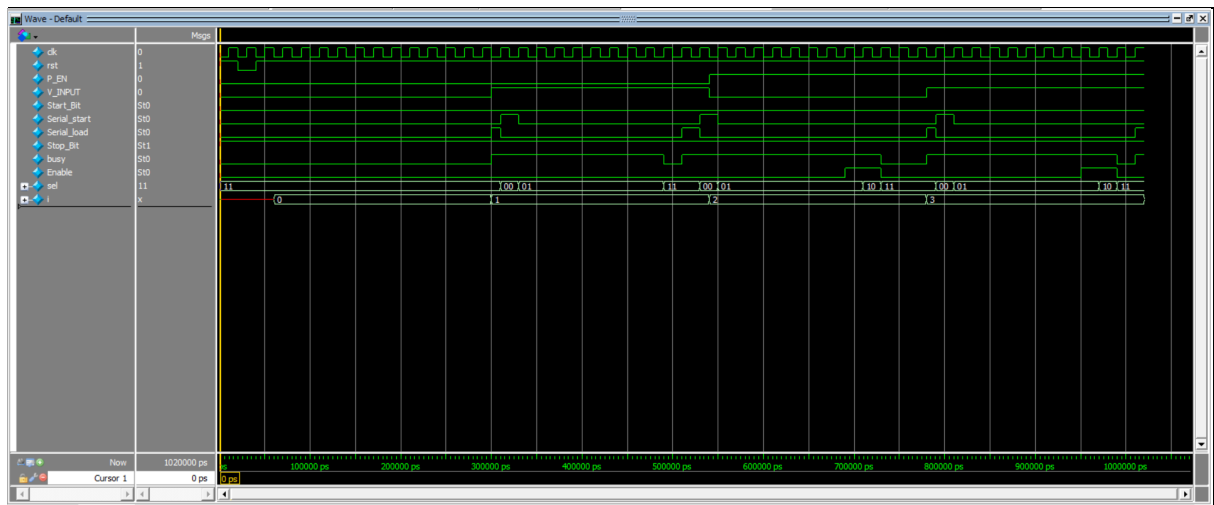
        {P_EN, V_INPUT} = i;

        repeat (12)
        begin
            $display ("P_EN = %b, V_INPUT = %b, Start_Bit = %b, Serial_start = %b, Serial_load = %b, Stop_Bit = %b, busy = %b, Enable = %b, sel = %d",
                    P_EN, V_INPUT, Start_Bit, Serial_start,
                    Serial_load, Stop_Bit, busy, Enable, sel);
            #20;
        end

        $display("End of Case");
    end

    $stop;
end
endmodule
```


Simulation Wave:



2. Parallel To Series Shift Register (Serializer):

This is the part that is responsible for converting the 8-bit parallel data to serial (one bit at a time)

The Verilog Code:

```

module Serializer (
input [7:0] P_INPUT,
input clk, rst, start, load,
output reg out
);

reg [2:0] counter;
reg [7:0] temp_input;
reg busy;

always @(posedge clk or negedge rst)
begin
    if (!rst)
        begin
            temp_input <= 0;
            busy <= 0;
            out <= 0;
            counter <= 3'd0;
        end

    else if (load && !busy)
        begin
            temp_input <= P_INPUT;
            counter <= 3'd0;
        end

    else if (start && !busy)
        begin
            busy <= 1;
            counter <= 3'd1;
            out <= temp_input[counter];
        end
end

```

```

        else if (busy)
        begin
            out <= temp_input[counter];
            counter <= counter + 3'd1;

            if (counter == 3'd7)
            begin
                busy <= 0;
            end
        end
    end
end
endmodule

```

Test Bench Code (Verilog):

```

`timescale 1ns/1ps

module Serializer_TB;

reg [7:0] P_INPUT;
reg clk, rst, start, load;
wire out;

Serializer dut ( .P_INPUT(P_INPUT), .clk(clk), .rst(rst), .start(start), .load(load), .out(out) );

integer i, j;

always #10 clk = ~clk;

task check;
input [7:0] P_INPUTch;

begin
    for (i = 0 ; i < 8 ; i = i + 1)
    begin
        if (out != P_INPUTch[i])
        begin
            $error ("Wrong Out = %d, P_INPUT[%d] = %d, P_INPUT = %b, Load = %b, Start = %b", out, i, P_INPUTch[i], P_INPUT, load, start);
        end

        else
        begin
            $display("Correct Out = %d, P_INPUT[%d] = %d, P_INPUT = %b, Load = %b, Start = %b", out, i, P_INPUTch[i], P_INPUT, load, start);
        end
        #20;
    end
end
endtask

initial
begin
    $dumpfile ("Serializer_TB.dmp");
    $dumpvars (0, Serializer_TB);

    clk = 0;
    rst = 1;
    start = 0;
    load = 0;
    P_INPUT = 0;

```

```

#40;
rst = 0;
#40;
rst = 1;

for (j = 0 ; j < 4 ; j = j + 1)
begin
    #20;

    load = 1; start = 0;
    P_INPUT = $random % 256;

    #20;

    {load, start} = j;

    #20;

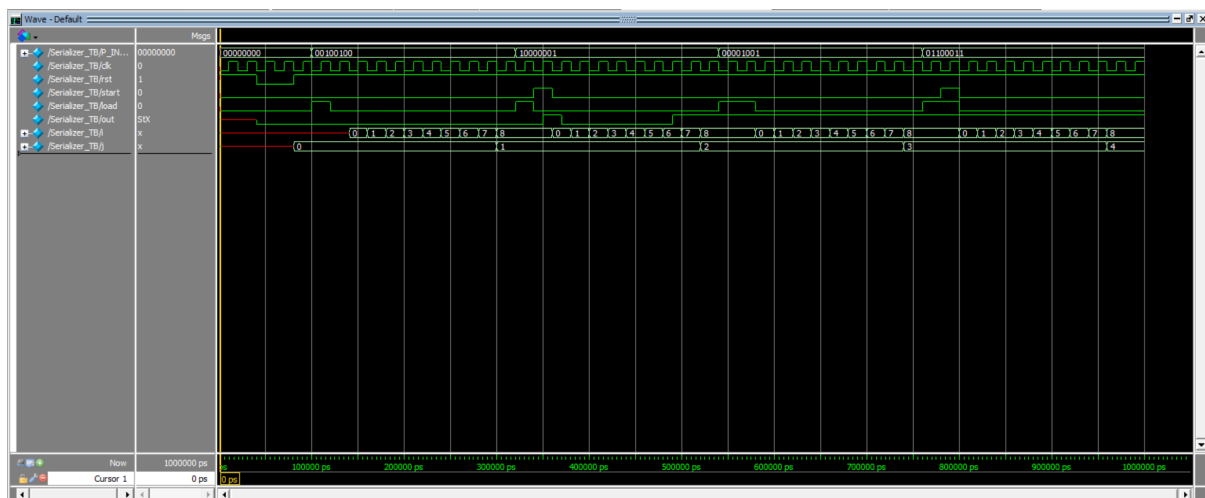
    load = 0; start = 0;

    check(P_INPUT);
    $display ("end of case");

end
#40;
$stop;
end
endmodule

```

Simulation Wave:



3. Parity Bit Calculator:

This module is responsible for checking the polarity of the Parallel data (If P_EN = 1)

How it works?

It simply uses XOR Gate as a comparator, since XOR outputs 1 if the number of high bits is odd and outputs 0 otherwise.

So we XOR every bit in the 8-bit data and if the P_BIT is 1 we send the current output otherwise we flip the output and send it.

What is it used for?

It is used as a data error check where it is present in both the TX and the RX. Both of them check the parity of the data and compare it between them, if there is a difference between them then there is a problem that need to be fixed.

The Verilog Code:

```
module Parity_Bit_Calculator (
input [7:0] P_INPUT,
input Enable, P_BIT,
output reg P_out
);

reg compare;

always @(*)
begin
    compare = P_INPUT[0] ^ P_INPUT[1] ^ P_INPUT[2] ^ P_INPUT[3] ^ P_INPUT[4] ^ P_INPUT[5] ^
P_INPUT[6] ^ P_INPUT[7];
    if (Enable)
    begin
        if (P_BIT)
        begin
            P_out = compare;
        end
        else
        begin
            P_out = !compare;
        end
    end
    else
    begin
        P_out = 0;
    end
end
endmodule
```

Test Bench (Verilog):

```
`timescale 1ns/1ps

module Parity_Bit_Calculator_TB;

reg [7:0] P_INPUT;
reg Enable, P_BIT;
wire P_out;

Parity_Bit_Calculator dut ( .P_INPUT(P_INPUT), .Enable(Enable), .P_BIT(P_BIT), .P_out(P_out) );

integer i, j;

function check;
input [7:0] P_INPUTch;
input Enablech, chP_BIT;
reg compare;

begin
```

```

        compare = 0;

        if (Enablech)
        begin
            for (i = 0 ; i < 8 ; i = i + 1)
            begin
                compare = compare ^ P_INPUTch[i];
            end
        end

        if (chP_BIT) check = compare;
        else check = !compare;
    end
endfunction

initial
begin
    $dumpfile ("Parity_Bit_Calculator_TB.dmp");
    $dumpvars (0, Parity_Bit_Calculator_TB);

    for (j = 0 ; j < 4 ; j = j + 1)
    begin
        #40;

        {Enable, P_BIT} = j;
        P_INPUT = $random % 256;

        #5;

        if (P_out == check(P_INPUT, Enable, P_BIT) && Enable)
        begin
            $display("Correct P_EN = %d, P_BIT = %d, P_INPUT = %b, out = %b", Enable,
P_BIT, P_INPUT, P_out);
        end

        else if (!Enable)
        begin
            $display("Ignored P_EN = %d, P_BIT = %d, P_INPUT = %b, out = %b", Enable,
P_BIT, P_INPUT, P_out);
        end

        else
        begin
            $error ("Wrong P_EN = %d, P_BIT = %d, P_INPUT = %b, out = %b", Enable, P_BIT,
P_INPUT, P_out);
        end
    end
end
$stop;
end
endmodule

```

Simulation Wave:



4. Mux:

This is the last module, and it is pretty simple actually. It takes all the outputs that we've talked about (Start bit, Serialized data, Parity Check, Stop bit) and takes a selector from the FSM and sends the data according to the selector.

Its implementation is just a case block with all the selector cases in it corresponding to the correct output

The Verilog Code:

```
module mux (  
input start, inbits, parity, stop,  
input [1:0] sel,  
output reg out  
);  
  
always @(*)  
begin  
    case (sel)  
        2'b00 : out = start;  
        2'b01 : out = inbits;  
        2'b10 : out = parity;  
        2'b11 : out = stop;  
    endcase  
end  
endmodule
```

Unfortunately, there is no test bench of Simulation waves since it is very simple for verification.

Now let's combine all of them and put them in our Top Module the TX. All we need to do is just instantiate the four previous modules and give them the required inputs and output.

The Verilog Code:

```
module UART_TX (  
input [7:0] P_INPUT,  
input V_INPUT, clk, rst, P_EN, P_BIT,  
output TX_OUTPUT, BUSY
```

```

);

wire [1:0] sel;
wire Start_Bit, Serial_start, Serial_load, Stop_Bit, Enable, Out1, Pout;

FSM_Controller FSMC ( .V_INPUT(V_INPUT), .clk(clk), .rst(rst), .P_EN(P_EN), .Start_Bit(Start_Bit),
.Serial_start(Serial_start),
                        .Serial_load(Serial_load), .Stop_Bit(Stop_Bit),
.Enable(Enable), .sel(sel), .busy(BUSY) );

Serializer Serial ( .P_INPUT(P_INPUT), .clk(clk), .rst(rst), .start(Serial_start), .load(Serial_load),
.out(Out1) );

Parity_Bit_Calculator Parity ( .P_INPUT(P_INPUT), .Enable(Enable), .P_BIT(P_BIT), .P_out(Pout) );

mux mux42 ( .start(Start_Bit), .inbits(Out1), .parity(Pout), .stop(Stop_Bit), .sel(sel),
.out(TX_OUTPUT) );

endmodule

```

Test Bench (Verilog):

```

`timescale 1ns/1ps

module UART_TX_TB;

reg [7:0] P_INPUT;
reg V_INPUT, clk, rst, P_EN, P_BIT;
wire TX_OUTPUT, BUSY;

UART_TX dut ( .P_INPUT(P_INPUT), .V_INPUT(V_INPUT), .clk(clk), .rst(rst), .P_EN(P_EN), .P_BIT(P_BIT),
.TX_OUTPUT(TX_OUTPUT), .BUSY(BUSY) );

integer i;

always #10 clk = ~clk;

initial
begin
    $dumpfile ("UART_TX_TB.dmp");
    $dumpvars (0, UART_TX_TB);

    clk = 0;
    rst = 1;
    P_EN = 0;
    V_INPUT = 0;
    P_BIT = 0;

    #20;

    for (i = 0 ; i < 8 ; i = i + 1)
    begin
        @(posedge clk);
        #1;
        {P_EN, P_BIT, V_INPUT} = i;
        P_INPUT = $random % 256;

        repeat (12)
        begin
            @(posedge clk);
            #1;
            $display ("P_EN = %b, P_BIT = %b, V_INPUT = %b, Busy = %b, P_INPUT = %b, Out = %b", P_EN, P_BIT, V_INPUT, BUSY, P_INPUT, TX_OUTPUT);

```



```

end

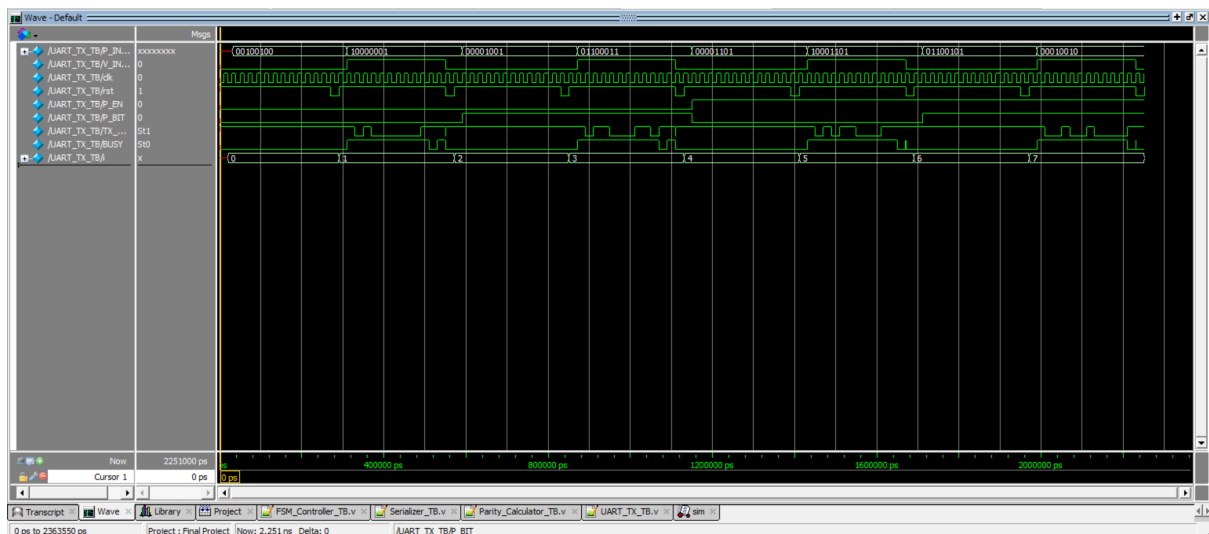
rst = 0;
V_INPUT = 0;
#20;
rst = 1;

$display("End of Case");

end
$stop;
end
endmodule

```

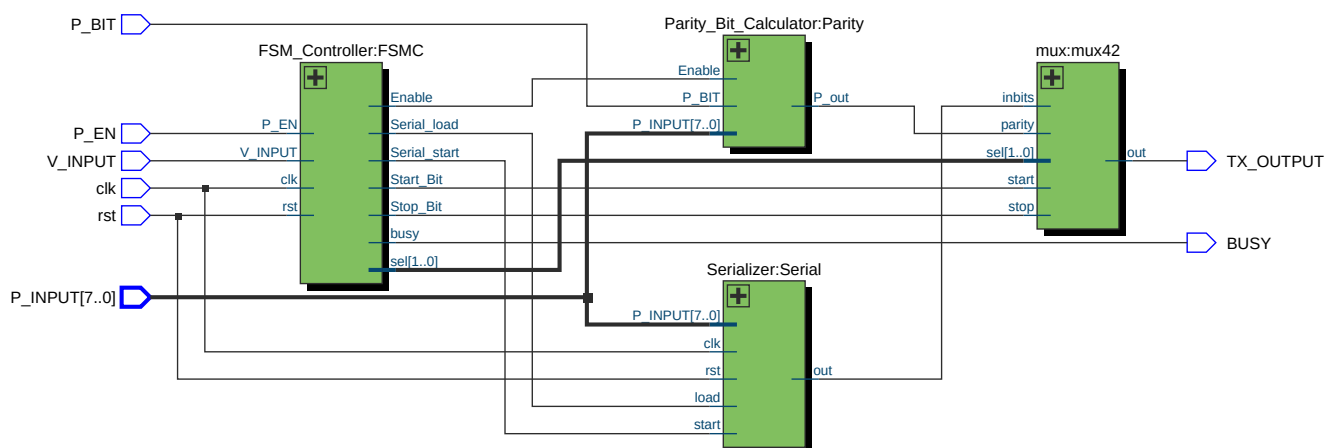
Simulation Waves:



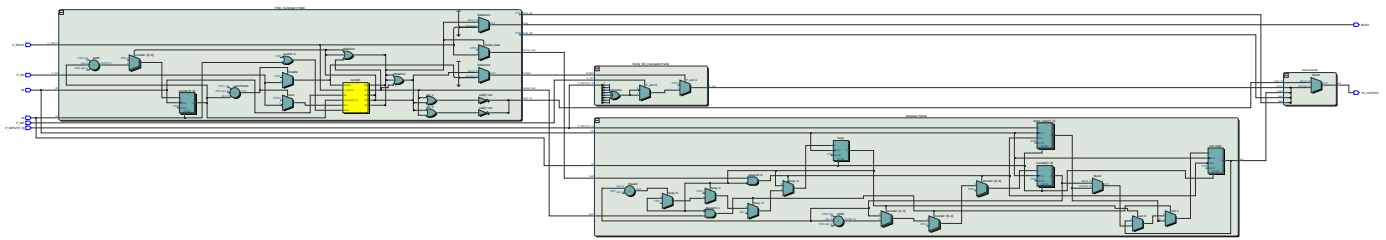
Extras (Schematics and time analysis from Quartus Prime):

After we have discussed about the TX and learned about its process and implementation in Verilog, here are some extra parts that also relates to the TX.

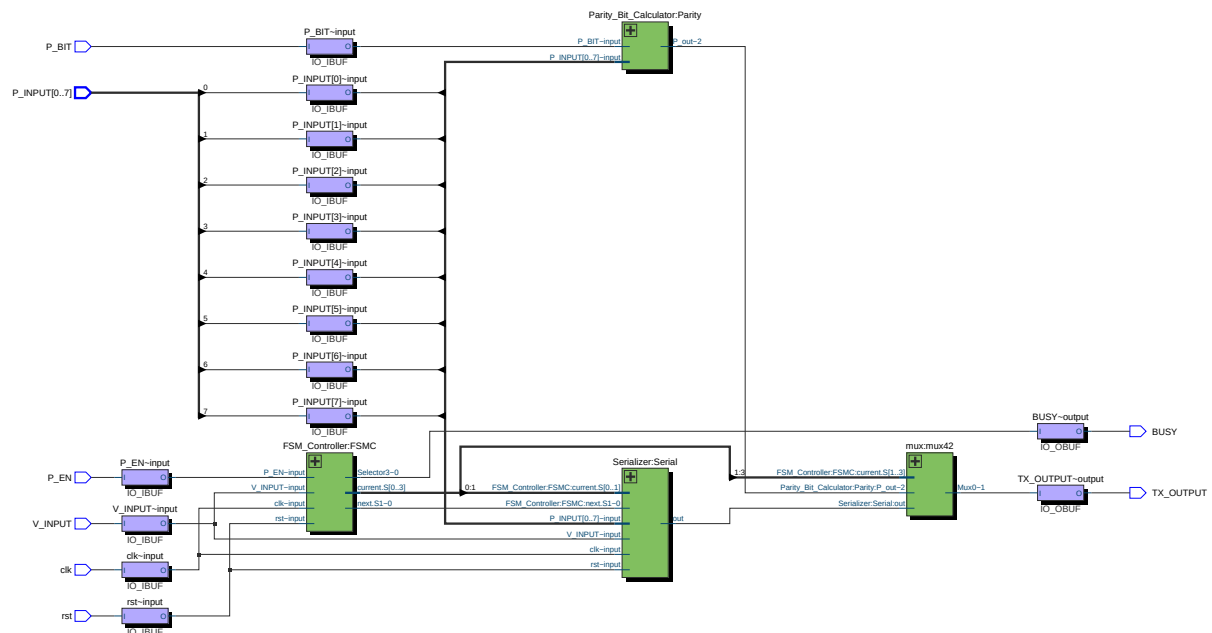
RTL VIEWER Schematic (Without Expanding):



RTL VIEWER Schematic (With Expanding):



Technology Map Power:



Time Analysis:

By setting the clock to 40ns (Frequency = 25Mhz), We achieve the following slacks:

Slow 1200 mv 85°C (Worst Case for Setup):

Setup				Hold			
	Clock	Slack	End Point TNS		Clock	Slack	End Point TNS
1	clk	26.114	0.000	1	clk	0.357	0.000

Slow 1200 mv 0°C:

Setup				Hold			
	Clock	Slack	End Point TNS		Clock	Slack	End Point TNS
1	clk	27.198	0.000	1	clk	0.310	0.000

Fast 1200 mv 0°C (Worst Case for Hold):

Setup				Hold			
	Clock	Slack	End Point TNS		Clock	Slack	End Point TNS
1	clk	29.873	0.000	1	clk	0.186	0.000

Timing Summary Table:

Operating Corner	Setup Slack	Hold Slack	Status
Slow 1200 mv 85°C	26.114 ns	0.357 ns	Pass
Slow 1200 mv 0°C	27.198 ns	0.310 ns	Pass
Fast 1200 mv 0°C	29.873 ns	0.186 ns	Pass

Worst-Case Slack:

	Clock	Setup	Hold	Recovery	Removal	Minimum Pulse Width
1	Worst-case Slack	26.114	0.186	N/A	N/A	19.437
1	clk	26.114	0.186	N/A	N/A	19.437

Conclusion: All timing requirements are met with positive slack across all corners, confirming the design will operate reliably from 0°C to 85°C.

Summary:

This project successfully implemented a fully functional UART transmitter. The design:

- ✓ Transmits 8-bit data in standard UART frame format.
- ✓ Supports configurable parity (Even / Odd / None).
- ✓ Operates with a frequency of 25 MHz system clock.
- ✓ Achieves timing closure across all Cases.
- ✓ Passes all functional verification tests.

Future Improvements:

1. Add a UART Receiver (RX) module.
2. Implement a 32-bit FIFO (First In First Out) buffer for extra data.
3. Support a wider range of bits (not just 8 bits).

And finally, I hope this project helped you understand how UART communication works and FPGA design, I am sure I did learn a lot, And I hope that I continue improving on this project!